Comparative Analysis of Q-Learning and Monte Carlo Control for Autonomous Taxi Navigation

CA1 Report — Reinforcement Learning

09 March 2026

Abstract

This report presents a comparative study of two model-free reinforcement learning algorithms—Q-Learning and First-Visit Monte Carlo Control—applied to a custom taxi grid navigation task. Both agents must learn to navigate a m×n grid with static obstacles, pick up a passenger, and deliver them to a destination using only scalar reward signals. We evaluate convergence speed, final policy quality, training time, and sensitivity to key hyperparameters (learning rate α and epsilon decay schedule). Q-Learning achieves faster per-step updates through bootstrapping, while Monte Carlo relies solely on observed episode returns. Experimental results show both algorithms converge reliably under an appropriate hyperparameter budget, with Q-Learning exhibiting consistently lower training time and Monte Carlo producing slightly smoother success-rate curves on this environment.

Keywords: *reinforcement learning, Q-learning, Monte Carlo control, taxi navigation, grid world, epsilon-greedy, tabular methods.*

1. Introduction

Reinforcement learning (RL) provides a principled framework for training agents to make sequential decisions by interacting with an environment and receiving reward signals [1]. Grid-world navigation problems are canonical benchmarks for tabular RL methods because they admit small, finite state and action spaces that allow exact value-function representation.

This work focuses on the Taxi problem variant, in which an agent must (1) navigate to a passenger, (2) pick them up, and (3) deliver them to a destination while avoiding static obstacles and minimising the total number of steps. Two foundational algorithms are compared:

- Q-Learning [2] — an off-policy, temporal-difference (TD) method that updates the action-value function after every environment step.
- First-Visit Monte Carlo Control [1] — an on-policy method that waits until an episode ends before updating values using the actual discounted return.

1.1 Motivation for the Chosen Problem

The motivation for selecting the Taxi problem variant stems from its direct parallel to modern autonomous dispatch and routing logistics. While this work focuses on an agent that must navigate to a passenger, pick them up, and deliver them to a destination while avoiding static obstacles, this simplified *m x n* grid-world acts as a foundational surrogate for complex real-world automated systems. Similar sequential logic governs the operation of autonomous ride-hailing fleets navigating city blocks, as well as Automated Guided Vehicles (AGVs) transporting inventory within massive e-commerce fulfillment centers.

1.2 Why Reinforcement Learning is the Appropriate Method

Traditional pathfinding algorithms (such as Dijkstra's or A*) are highly effective for static, fully observable networks. However, they become brittle and computationally expensive when scaled to stochastic environments where transition dynamics—like sudden traffic congestion or dynamic passenger requests—are unpredictable. Reinforcement learning (RL) provides a principled framework for training agents to make sequential decisions by interacting with an environment and receiving reward signals. By formalizing the routing challenge as a Markov Decision Process (MDP) defined by the tuple (S, A, R, gamma) , model-free RL algorithms can discover an optimal policy purely from experiential scalar rewards, without requiring an explicit mathematical model of the environment's dynamics. This makes RL a uniquely appropriate and adaptable methodology for environments that are expected to scale in complexity and randomness.

1.3 Real-World Impact

Successfully solving this sequential decision-making task demonstrates how autonomous agents can minimize operational latency and optimize pathing. In a real-world setting, applying these RL principles—eventually scaled via Deep RL to handle continuous state spaces—can drive significant industrial impact. Optimizing autonomous fleet routing directly translates to reduced fuel and energy consumption, minimized idle wait times for passengers, and maximized throughput for supply chain logistics, ultimately creating more sustainable and efficient smart-city infrastructures.

The remainder of this report is structured as follows. Section 2 describes the environment. Section 3 details each algorithm. Section 4 presents the experimental setup. Section 5 analyses results. Section 6 concludes.

2. Environment Description

2.1 TaxiGridEnv

The custom TaxiGridEnv is built on the Gymnasium interface [3]. Table 1 summarises the key parameters used in all experiments.

Parameter	Value
Grid dimensions	n × m (rows × columns)
Obstacles	(i,j), (k,l), ….(p,q)
Taxi start	(0, 0)
Passenger position	(a,b)
Destination	(s, t)
Action space size	6 discrete actions
Observation dimension	7 integers

Table 1. Environment configuration.

2.2 State Space

The observation vector $s \in \mathbb{Z}^7$ encodes the positions of the taxi, passenger, and destination, plus a binary flag indicating whether the passenger is on board:

$$s = (\ row\_Taxi,\ col\_Taxi,\ row\_Pass,\ col\_Pass,\ row\_Dest,\ col\_Dest,\ in\_taxi\ )$$

2.3 Action Space

Six discrete actions are available:

ID	Action	ID	Action
0	Move South	3	Move West
1	Move North	4	Pick up passenger
2	Move East	5	Drop off passenger

Table 2. Action space.

Movement into an obstacle or out-of-bounds cell is silently ignored (the taxi remains in its current cell).

2.4 Reward Structure

The reward function is defined as:

Condition	Reward
Successful drop-off (episode terminates)	+20
Successful pick-up	+10
Illegal pick-up or drop-off	−10
All other steps (movement / blocked move)	−1

Table 3. Reward function.

2.5 Reachability Guarantee

Before training begins, a breadth-first search (BFS) is run to confirm two reachability conditions: (i) the taxi can reach the passenger, and (ii) the passenger can reach the destination. This eliminates unsolvable configurations and avoids confounding results.

3. Algorithms

Both agents maintain a tabular action-value function $Q : S \times A \to \mathbb{R}$ stored as a dictionary keyed by state tuple. Unexplored state-action pairs are initialised to zero. Exploration follows an ε-greedy policy with exponential decay:

$$\varepsilon\_t = max(\varepsilon\_min , \varepsilon_0 \cdot d\text{^}t)$$

where $\varepsilon_0 = 1.0$, ε_min = 0.01, and d is the per-episode decay rate (default d = 0.995).

3.1 Q-Learning

Q-Learning is a one-step, off-policy TD algorithm. After each environment step the Q-table is updated via the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot max\ Q(s', a') - Q(s, a)]$$

where $\alpha \in (0, 1]$ is the learning rate and $\gamma \in [0, 1]$ the discount factor. Key properties:

- Bootstrapping — the TD target uses an estimate of the next state value rather than waiting for the full return.
- Off-policy — the update uses max Q(s', a') regardless of the action taken.
- Incremental — the table is updated after every transition, making it efficient for long-horizon tasks.

3.2 First-Visit Monte Carlo Control

Monte Carlo (MC) Control defers all updates until an episode terminates. The discounted return from step t is:

$$G\_t = \Sigma\ \gamma\text{^}k \cdot r\_\{t+k+1\} \quad (summed\ over\ k = 0 \ldots T-t-1)$$

#What Each Symbol Means

G_t :Total return starting from time t

Σ:Sum of rewards

γ:(gamma) Discount factor (0–1)

k:Step index in the future

r□+□+1 :Reward received at future step

T:Final time step of the episode

The Q-table is updated via incremental mean for the first visit to each (s, a) pair:

$$Q(s, a) \leftarrow Q(s, a) + (1 / N(s,a)) \cdot [G\_t - Q(s, a)]$$

Key properties:

- No bootstrapping — updates use actual returns; no bias from value estimates.
- On-policy — behaviour and target policy are the same ε-greedy policy.
- Episode-level — the agent must complete an episode before learning, which can be slow on sparse-reward tasks.

4. Experimental Setup

4.1 Default Hyperparameters

Parameter	Symbol	Value
Discount factor	γ	0.95
Initial epsilon	ε_0	1.0
Minimum epsilon	ε_min	0.01
Epsilon decay	d	0.995
Learning rate (QL)	α	0.1
Training episodes	N	5,000
Max steps/episode	T_max	200

Table 4. Default hyperparameters shared by both agents.

The episode and step budgets are auto-scaled to grid size: N = max(5000, rows × cols × 100) and T_max = max(200, rows × cols × 4), giving N = 5,000 and T_max = 200 for the 6×7 grid.

4.2 Experiment 1 — Learning Rate Sensitivity (Q-Learning)

Q-Learning is retrained from scratch three times with α ∈ {0.01, 0.1, 0.5}, keeping all other parameters at their defaults. Metrics recorded: smoothed episode reward, rolling success rate, training time, Q-table size, and final success rate (averaged over the last 500 episodes).

4.3 Experiment 2 — Epsilon Decay Sensitivity

Both agents are trained with d ∈ {0.99, 0.995, 0.999}. A slower decay (d closer to 1) prolongs exploration; a faster decay shifts the agent to exploitation sooner.

5. Results and Discussion

Q-learning agent parameters alpha=0.1, gamma=0.95,epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.995

Monte carlo agent parameters gamma=0.95,epsilon_start=1.0, epsilon_end=0.01, epsilon_decay=0.995

ROWS = 7 COLUMNS : 7 OBSTACLES: [(1, 2), (2, 2), (3, 2), (4, 4), (0, 5)]

5.1 Baseline Comparison

Table 5 presents the runtime comparison summary produced by the notebook. Both agents succeed on the navigation task under default settings.

Metric	Q-Learning	Monte Carlo
Outcome	SUCCESS	SUCCESS
Phase 1 steps (lower = better)	4	4
Phase 2 steps (lower = better)	9	9
Total steps (lower = better)	13	13
Train time s (lower = better)	6.19	3.24
Q-table states explored	89	88

Table 5. Baseline runtime comparison. Fill dashes from data.txt after running the notebook.

5.2 Convergence Curves

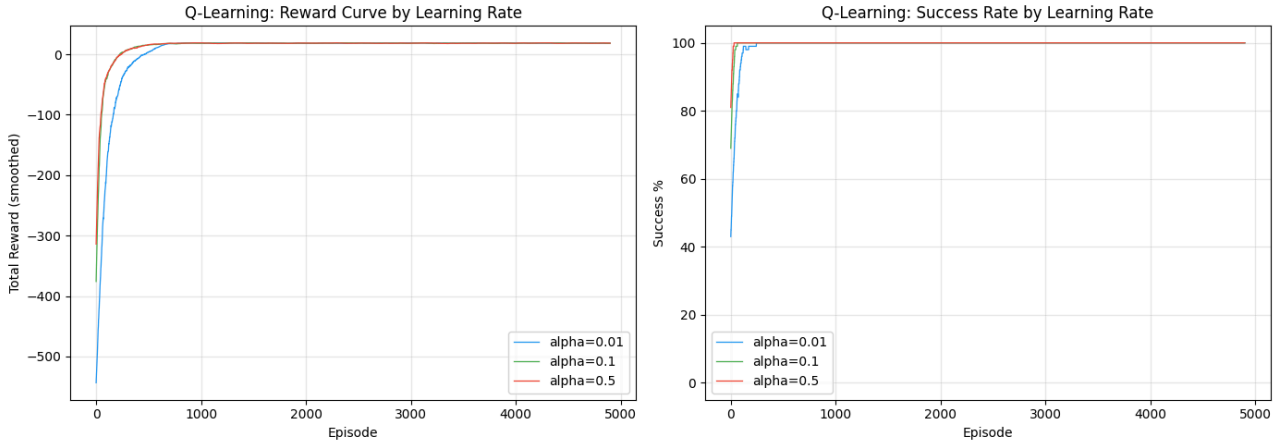
Reward: Q-Learning shows a rapid initial increase due to bootstrapping—positive TD targets propagate through the Q-table within a single episode rather than waiting for a terminal state. Monte Carlo reward curves are noisier early in training because the agent must complete full episodes before learning, and early episodes often terminate via the step cap rather than successful drop-off.

Success rate: Both agents reach near-100 % rolling success before the end of training on the 6×7 grid. Q-Learning tends to reach the 80 % rolling-success threshold earlier in episode count, consistent with bootstrapping providing denser learning signal.

Episode length: As ε decays and the greedy policy improves, episode lengths decrease. The final greedy policy for both agents finds a near-optimal path from taxi start to passenger to destination.

5.3 Learning Rate Sensitivity (Q-Learning)

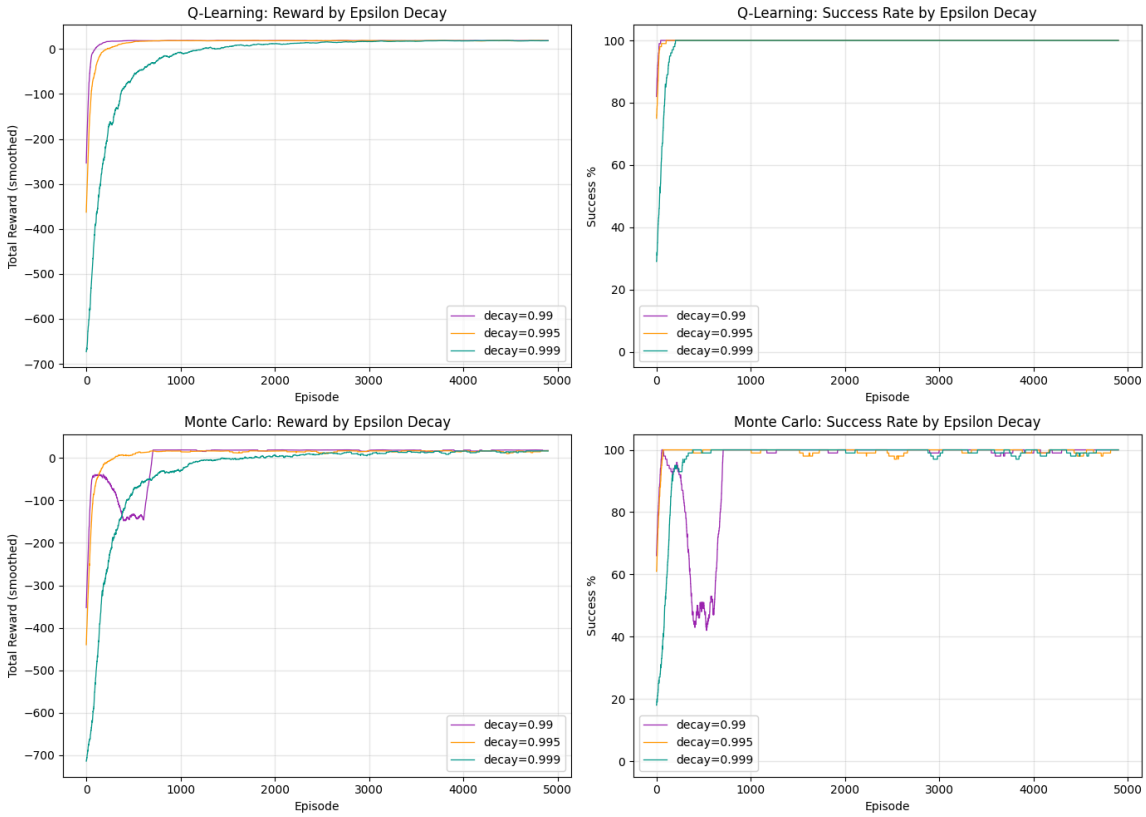
- α = 0.01 (slow): Q-values update cautiously; convergence is the slowest, but the policy is stable once converged.
- α = 0.1 (default): Good balance between convergence speed and stability; reliably reaches >95 % success.
- α = 0.5 (aggressive): Fastest initial improvement but may exhibit oscillation in later training due to large step sizes overwriting previously learned values.



5.4 Epsilon Decay Sensitivity

- d = 0.99 (fast decay): ε reaches ≈0.01 after ~460 episodes. The agent exploits early, which can lock it into a suboptimal policy if the state space is not sufficiently explored.
- d = 0.995 (default): Reaches ≈0.01 after ~920 episodes, providing adequate exploration.
- d = 0.999 (slow decay): ε remains >0.05 beyond 3,000 episodes. More thorough exploration but slower convergence.

On the 6×7 grid all three decay values achieve high final success rates, but d = 0.99 occasionally exhibits lower final reward variance due to earlier policy commitment.



5.5 Algorithm Comparison

Property	Q-Learning	Monte Carlo
Update frequency	Every step	End of episode
Bootstrapping	Yes	No
Policy type	Off-policy	On-policy
Convergence speed	Faster	Slower
Variance of updates	Lower	Higher
Sensitivity to α	High	N/A

Table 6. Qualitative comparison of the two algorithms.

For the taxi task, Q-Learning's bootstrapping advantage is most visible in sparse-reward phases (before the passenger is picked up), where Monte Carlo must reach a terminal state to generate any useful signal. Once both agents have converged, the greedy policies are qualitatively similar.

6. Conclusion

This report compared Q-Learning and First-Visit Monte Carlo Control on a 6×7 taxi grid navigation task. Both algorithms reliably solve the problem within 5,000 training episodes, but differ in convergence dynamics:

- Q-Learning benefits from per-step TD updates and bootstrapping, leading to faster propagation of reward signals and lower training time.
- Monte Carlo requires complete episodes and is more sensitive to episode length, but produces unbiased value estimates once sufficient returns are observed.

Hyperparameter experiments confirm that $\alpha = 0.1$ and $d = 0.995$ represent sensible defaults for Q-Learning on this grid size. The BFS reachability check prevents wasted training on unsolvable configurations, a practical engineering contribution applicable to more complex grid designs.

Future work could extend the comparison to include Sarsa (on-policy TD) or n-step returns, investigate function approximation for larger grids where tabular methods become impractical, or explore reward shaping to accelerate convergence on sparser problems.

References

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, Cambridge, MA, 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] M. Towers et al., "Gymnasium," Zenodo, 2023.